Interfacing Keyboard

Interfacing a Microprocessor To Keyboard

- When you press a key on your computer, you are activating a switch. There are many different ways of making these switches.
    1. Mechanical key switches
    2. Membrane key switches
    3. Capacitive key switches
    4. Hall effect key switches

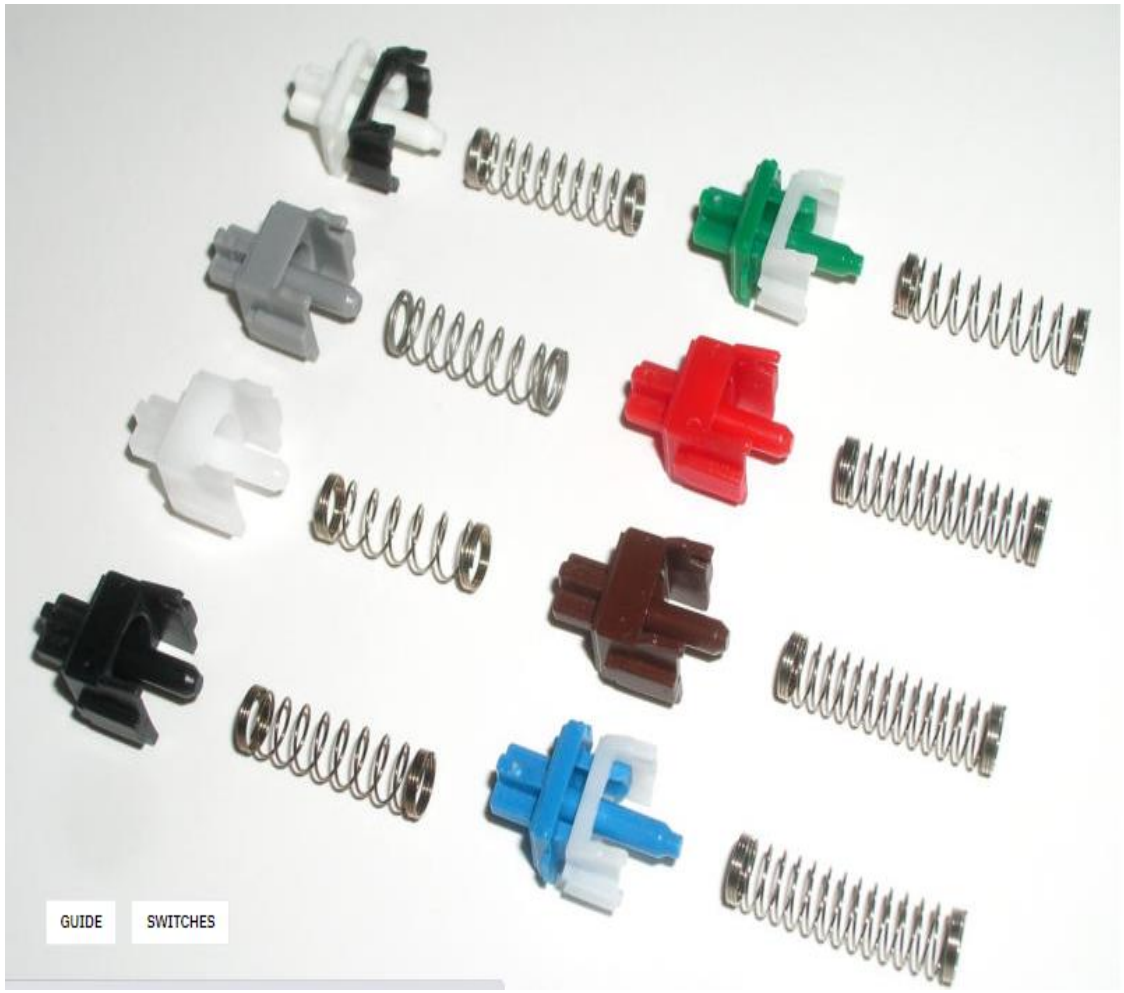
Mechanical key switches

- In mechanical-switch keys, two pieces of metal are pushed together when you press the key.
- The actual switch elements are often made of a phosphor-bronze alloy with gold platting on the contact areas.
- The key switch usually contains a spring to return the key to the non-pressed position and perhaps a small piece of foam to help damp out bouncing.
- Some mechanical key switches now consist of a molded silicon dome with a small piece of conductive rubber on the underside.
- When a key is pressed, the rubber foam shorts two traces on the printed-circuit board to produce the key pressed signal.

Mechanical key switches

- Mechanical switches are relatively inexpensive but they have several disadvantages.
  - First, they suffer from contact bounce. A pressed key may make and break contact several times before it makes solid contact.
  - Second, the contacts may become oxidized or dirty with age so they no longer make a dependable connection.
- Higher-quality mechanical switches typically have a rated life time of about 1 million keystrokes.
- The silicone dome type typically last 25 million keystrokes.

Mechanical key switches



Membrane key switches

- These switches are really a special type of mechanical switches. They consist of a three-layer plastic or rubber sandwich.
- The top layer has a conductive line of silver ink running under each key position.
- The bottom layer has a conductive line of silver ink running under each column of keys.
- When u press a key, you push the top ink line through the hole to contact the bottom ink line.
- The advantages of membrane keyboards is that they can be made as very thin, sealed units.
- They are often used on cash registers in fast food restaurants. The lifetime of membrane keyboards varies over a wide range.

Capacitive key switches:

- A capacitive key switch has two small metal plates on the printed circuit board and another metal plate on the bottom of a piece of foam.
- When u press the key, the movable plate is pushed closer to fixed plate. This changes the capacitance between the fixed plates.
- Sense amplifier circuitry detects this change in capacitance and produce a logic level signal that indicates a key has been pressed.
- The big advantages of a capacitive switch is that it has no mechanical contacts to become oxidized or dirty.
- A small disadvantage is the specified circuitry needed to detect the change in capacitance.
- Capacitive key switches typically have a rated lifetime of about 20 million keystrokes.

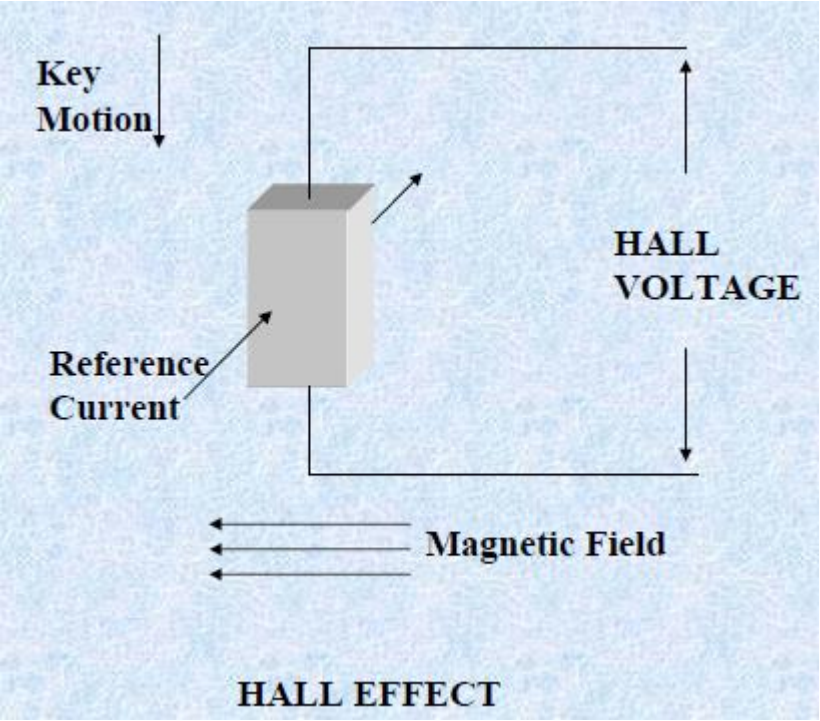
Hall effect key switches

- This is another type of switch which has no mechanical contact.
- It takes advantage of the deflection of a moving charge by a magnetic field.
- A reference current is passed through a semiconductor crystal between two opposing faces.
- When a key is pressed, the crystal is moved through a magnetic field which has its flux lines perpendicular to the direction of current flow in the crystal.
- Moving the crystal through the magnetic field causes a small voltage to be developed between two of the other opposing faces of the crystal.

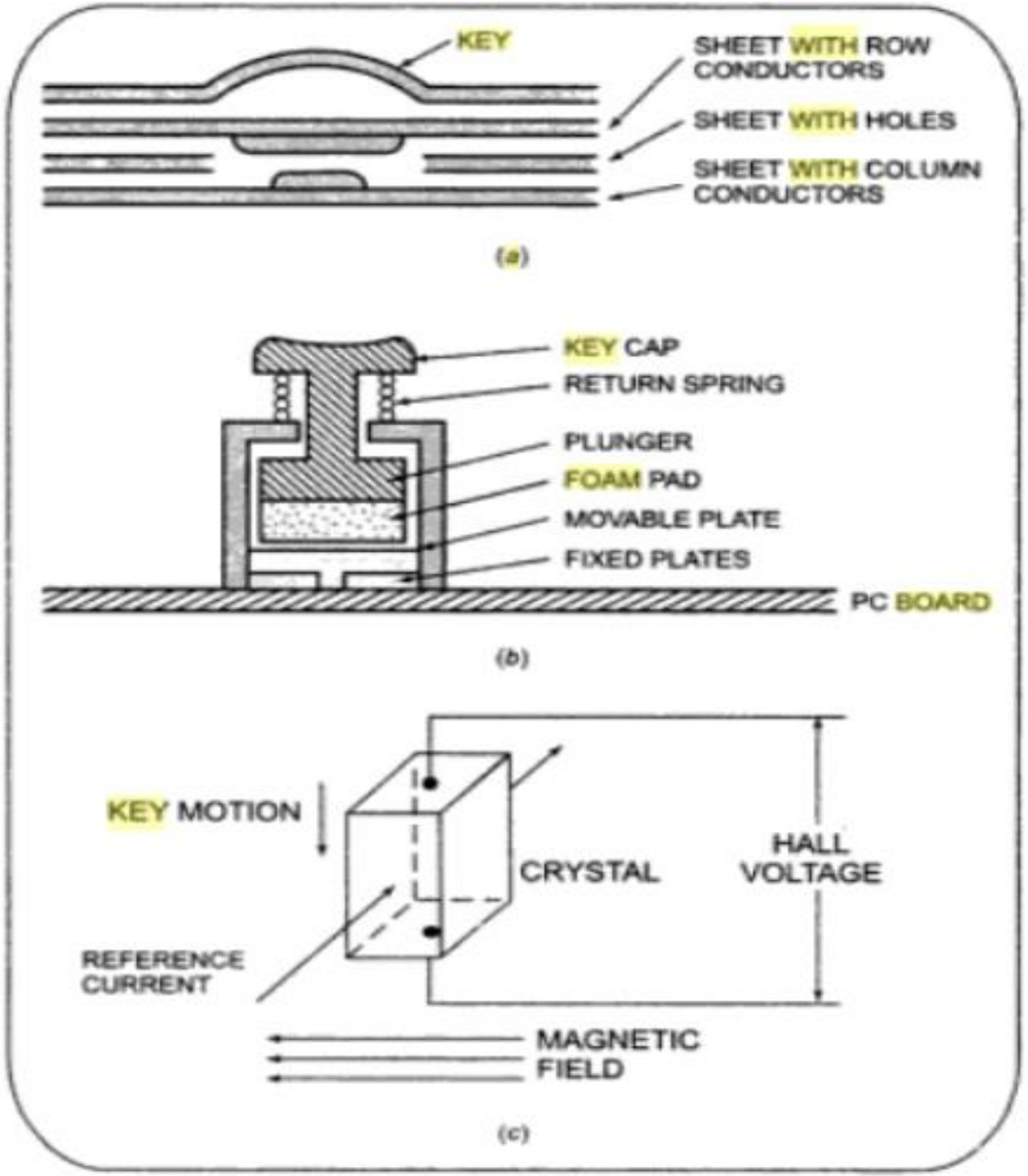
Hall effect key switches

- This voltage is amplified and used to indicate that a key has been pressed.
- Hall effect sensors are also used to detect motion in many electrically controlled machines.
- Hall effect keyboards are more expensive because of the more complex switch mechanism, but they are very dependable and have typically rated lifetime of 100 million or more keystrokes.

Hall effect key switches



a. Membrane
b. Capacitive
c. Hall-effect



Keyboard Circuit Connections and Interfacing

- In most keyboards, the key switches are connecting in a matrix of rows and columns.
- We will use simple mechanical switches for our examples, but the principle is same for other type of switches.
- Getting meaningful data from a keyboard, it requires the following three major tasks:
    1. Detect a keypress
    2. Debounce the keypress
    3. Encode the keypress

Keyboard Circuit Connections and Interfacing

- Three tasks can be done with hardware, software, or a combination of two, depending on the application.

1. Software Keyboard Interfacing:

- ***Circuit connection and algorithm :***
  - The rows of the matrix are connected to four output port lines.
  - The column lines of matrix are connected to four input-port lines.
  - To make the program simpler, the row lines are also connected to four input lines.

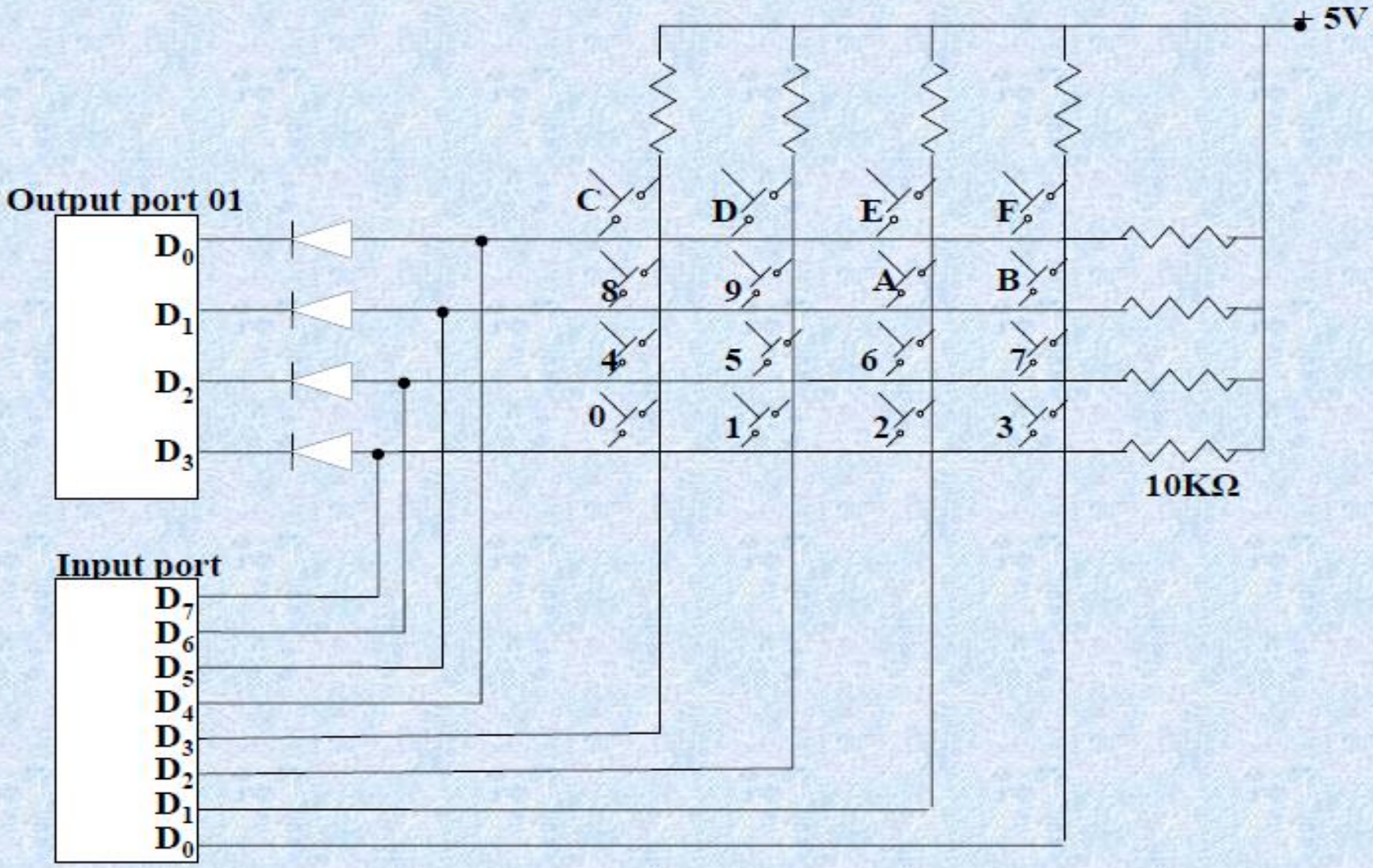


Fig: (a) Port connections

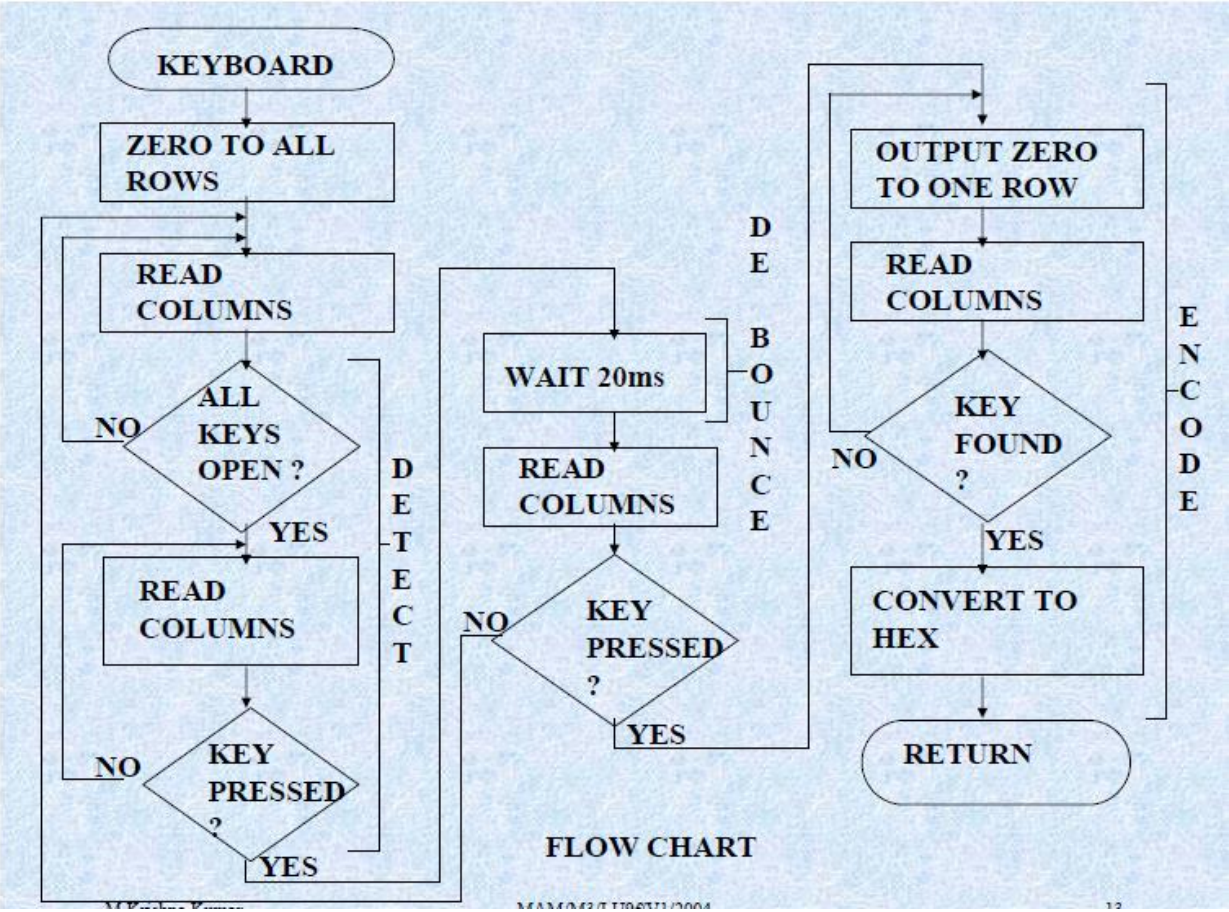
Software Keyboard Interfacing

- When no keys are pressed, the column lines are held high by the pull-up resistor connected to +5V.
- Pressing a key connects a row to a column.
- If a low is output on a row and a key in that row is pressed, then the low will appear on the column which contains that key and can be detected on the input port.
- If you know the row and column of the pressed key, you then know which key was pressed, and you can convert this information into any code you want to represent that key.

Software Keyboard Interfacing

- An easy way to detect if any key in the matrix is pressed is to output 0's to all rows and then check the column to see if a pressed key has connected a low to a column.
- In the algorithm, we first output lows to all the rows and check the columns over and over until the column are all high.
- This is done before the previous key has been released before looking for the next one.
- In the standard keyboard terminology, this is called two-key lockout.

Software Keyboard Interfacing



Software Keyboard Interfacing

- Once the columns are found to be all high, the program enters another loop, which waits until a low appears on one of the columns, indicating that a key has been pressed.
- This second loop does the detect task for us.
- A simple 20-ms delay procedure then does the debounce task.
- After the debounce time, another check is made to see if the key is still pressed.
- If the columns are now all high, then no key is pressed and the initial detection was caused by a noise pulse or a light brushing past a key.
- If any of the columns are still low, then the assumption is made that it was a valid keypress.

Software Keyboard Interfacing

- The final task is to determine the row and column of the pressed key and convert this row and column information to the hex code for the pressed key.
- To get the row and column information, a low is output to one row and the column are read.
- If none of the columns is low, the pressed key is not in that row.
- So the low is rotated to the next row and the column are checked again.
- The process is repeated until a low on a row produces a low on one of the column.
- The pressed key then is in the row which is low at that time.

Software Keyboard Interfacing

- This code would not match any of the row-column codes in the table, so after all the values in the table were checked, assigned register in program would be decremented from 0000H to FFFFH.
- The compare decrement cycle would continue through 65,536 memory locations until, by change the value in a memory location matched the row-column code.
- The contents of the lower byte register at that point would be passed back to the calling routine.
- The changes are 1 in 256 that would be the correct value for one of the pressed keys.
- You should keep an error trap in a program whenever there is a chance for it.

Keyboard Interfacing with Hardware

2. Keyboard Interfacing with Hardware:

- For the system where the CPU is too busy to be bothered doing these tasks in software, an external device is used to do them.
- One of a MOS device which can be do this is the General Instruments AY5-2376 which can be connected to the rows and columns of a keyboard switch matrix.
- The AY5-2376 independently detects a keypress by cycling a low down through the rows and checking the columns.
- When it finds a key pressed, it waits a debounce time.

Keyboard Interfacing with Hardware

- If the key is still pressed after the debounce time, the AY5- 2376 produces the 8-bit code for the pressed key and send it out to microcomputer port on 8 parallel lines.
- The microcomputer knows that a valid ASCII code is on the data lines, the AY5-2376 outputs a strobe pulse.
- The microcomputer can detect this strobe pulse and read in ASCII code on a polled basis or it can detect the strobe pulse on an interrupt basis.
- With the interrupt method the microcomputer doesn’t have to pay any attention to the keyboard until it receives an interrupt signal.

Keyboard Interfacing with Hardware

- So this method uses very little of the microcomputer time.
- The AY5-2376 has a feature called *two-key rollover.*
- This means that if two keys are pressed at nearly the same time, each key will be detected, debounced and converted to ASCII.
- The ASCII code for the first key and a strobe signal for it will be sent out then the ASCII code for the second key and a strobe signal for it will be sent out and compare this with two-key lockout.

Keyboard Interfacing with Hardware

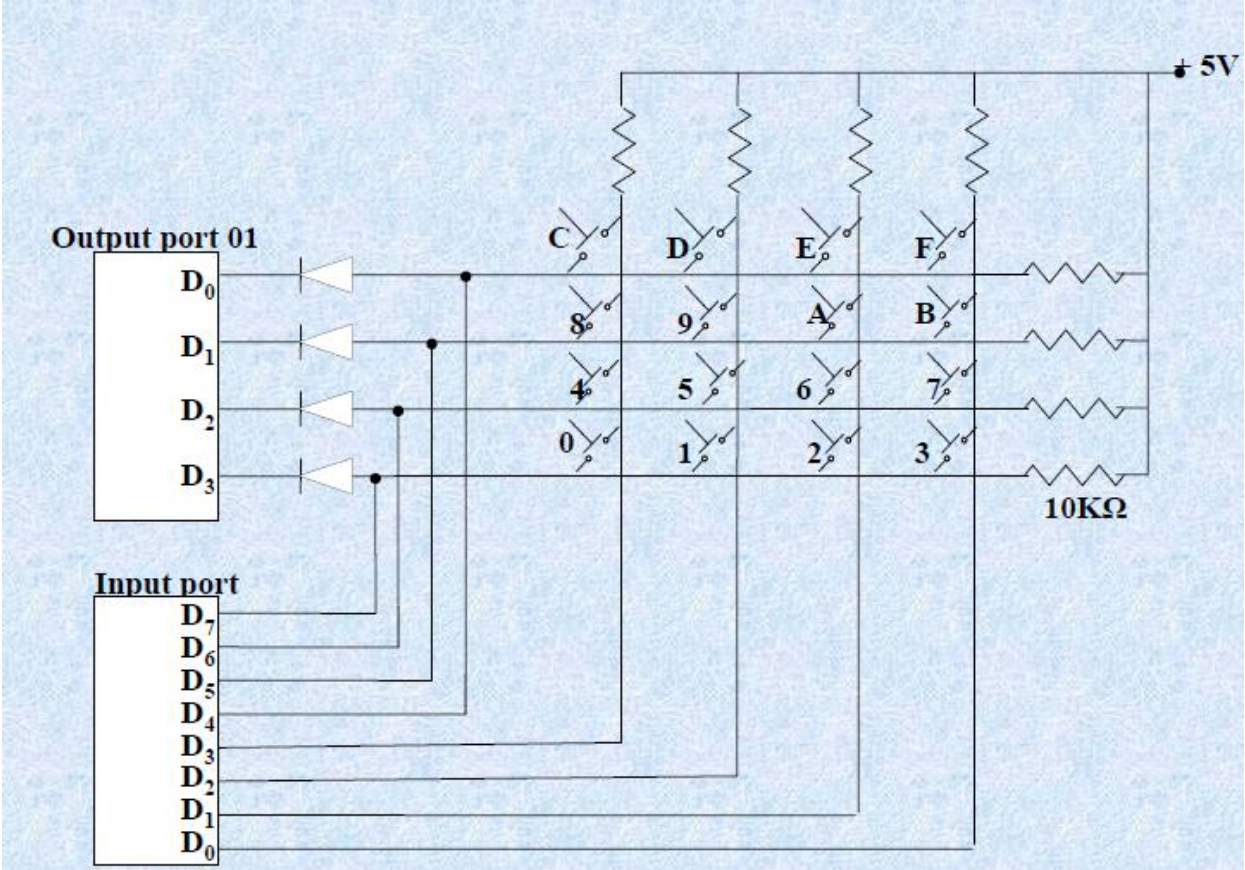


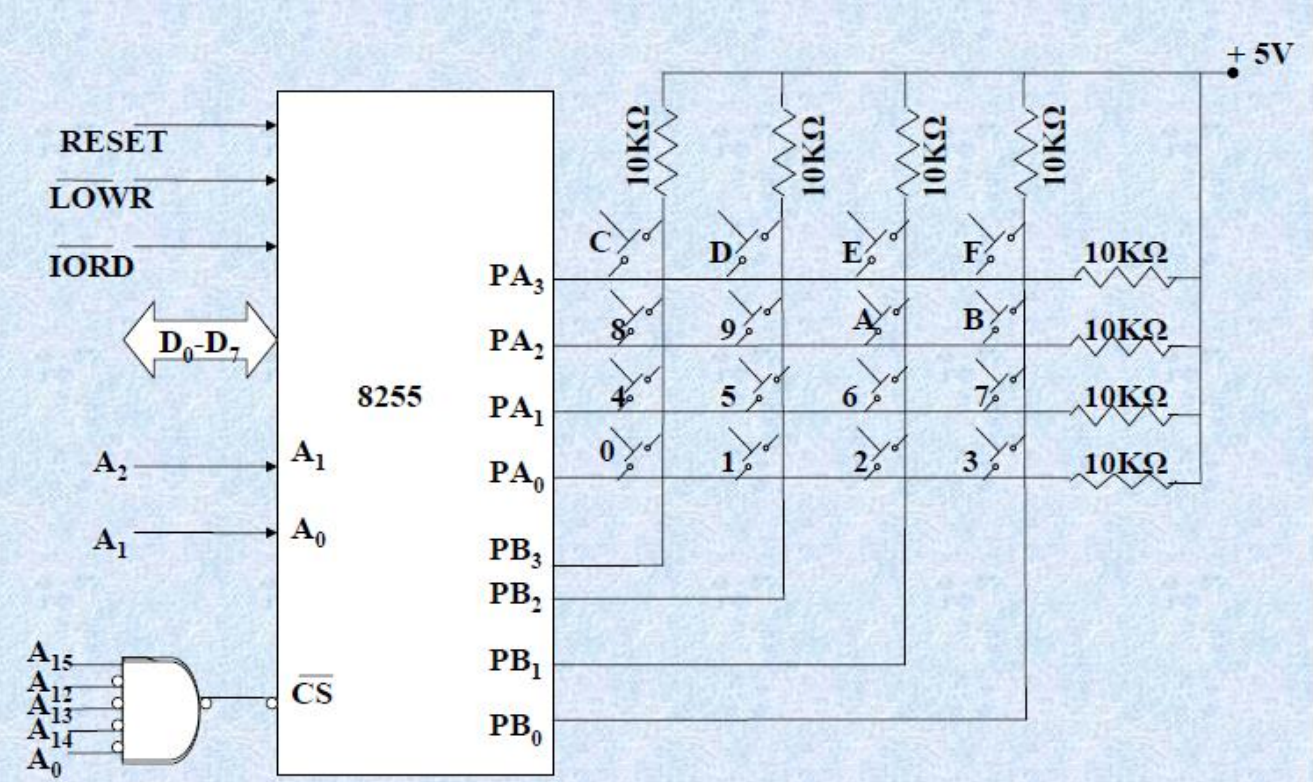
Fig: (a) Port connections

Example

- Interface a 4 * 4 keyboard with 8086 using 8255 and write an ALP for detecting a key closure and return the key code in AL. The debounce period for a key is 10ms. Use software debouncing technique. DEBOUNCE is an available 10ms delay routine.

- **Solution**: Port A is used as output port for selecting a row of keys while Port B is used as an input port for sensing a closed key.
- Thus the keyboard lines are selected one by one through port A and the port B lines are polled continuously till a key closure is sensed.
- The routine DEBOUNCE is called for key debouncing.
- The key code is depending upon the selected row and a low sensed column.

Keyboard Interfacing with Hardware

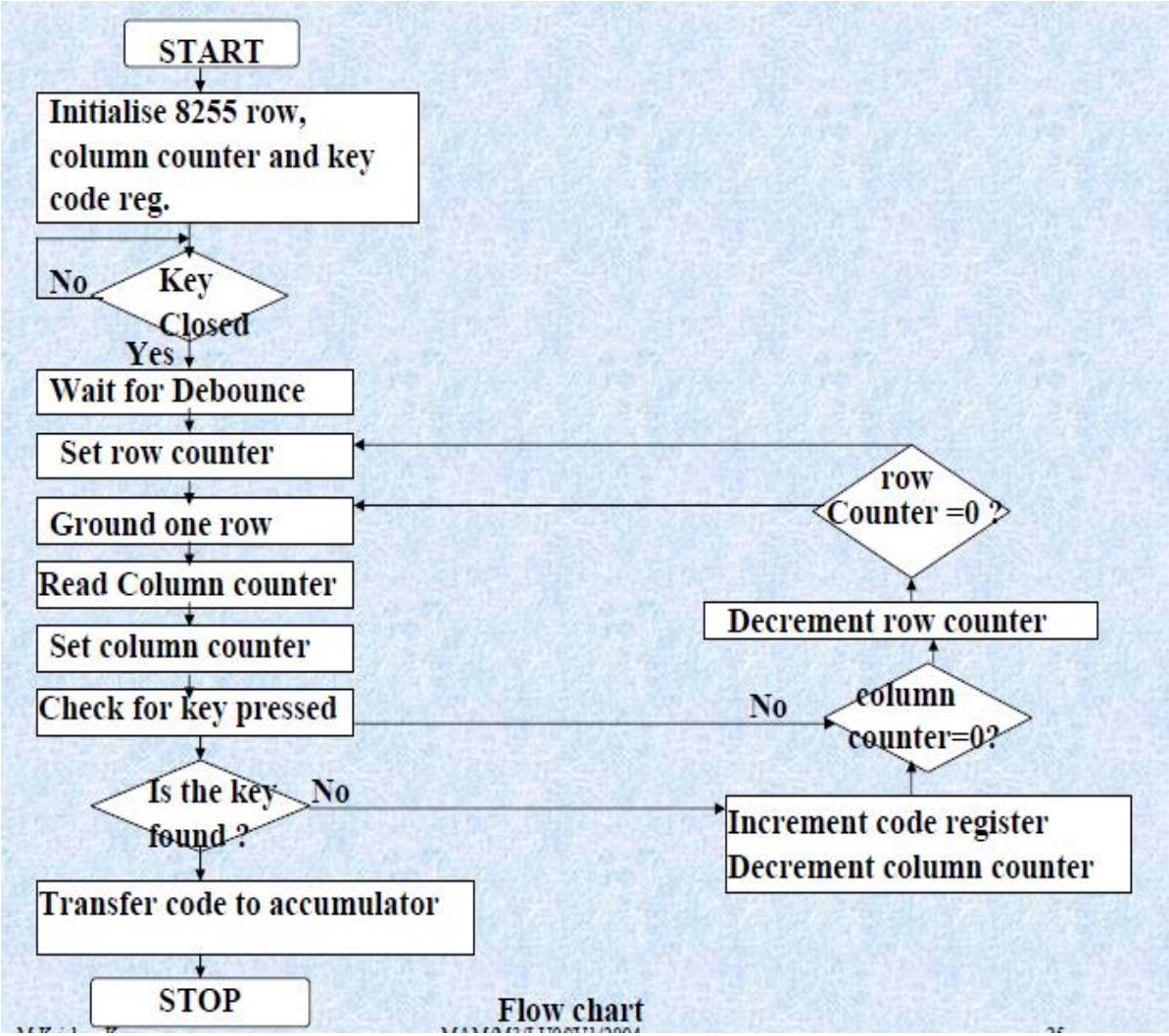


Interfacing 4 * 4 Keyboard

Keyboard Interfacing with Hardware

- The higher order lines of port A and port B are left unused.
- The address of port A and port B will respectively 8000H and 8002H while address of CWR will be 8006H.
- The control word for this problem will be 82H.
- Code segment CS is used for storing the program code.
- **Key Debounce** : Whenever a mechanical push-button is pressed or released once, the mechanical components of the key do not change the position smoothly, rather it generates a transient response .

Flow Chart

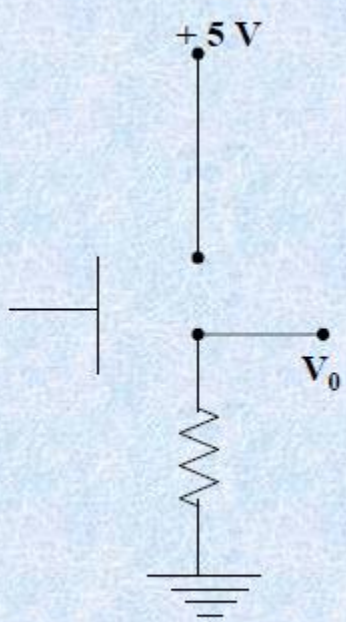


Flow chart

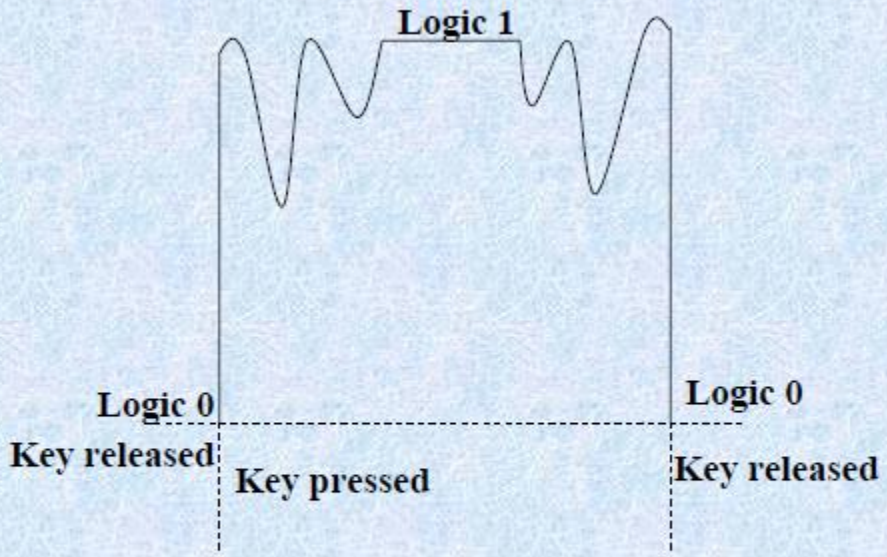
- These transient variations may be interpreted as the multiple key pressure and responded accordingly by the microprocessor system.
- To avoid this problem, two schemes are suggested:
  - The first one utilizes a bistable multivibrator at the output of the key to debounce .
  - The other scheme suggests that the microprocessor should be made to wait for the transient period (usually 10ms), so that the transient response settles down and reaches a steady state.

- A logic ‘0’ will be read by the microprocessor when the key is pressed.
- In a number of high precision applications, a designer may have two options- the first is to have more than one 8-bit port, read (write) the port one by one and then from the multibyte data, the second option allows forming 16-bit ports using two 8-bit ports and use 16-bit read or write operations.

Example



A Mechanical Key



Response

Defining CWR , Port A and Port B of 8255

8255 PPI Programming							
Control Word Register (0C6H)							
8				2			
1	0	0	0	0	0	1	0
I/O	MODE - 0		PORT A		MODE - 0	PORT B	
	PORT A		O/P		PORT B	I/P	

```
ROW_0 DB 0, 1, 2, 3
ROW_1 DB 4, 5, 6, 7
ROW_2 DB 8, 9, 0AH, 0BH
ROW_3 DB 0CH, 0DH, 0EH, 0FH
KEY DB 0

    CR EQU 0C6H      ;address of the control register
    PA EQU 0C0H      ;address of Port A
    PB EQU 0C2H      ;address of Port B

    MOV AL, 82H      ;8255 control word
    OUT CR, AL       ;send to control register
```

Activate Windows
Go to Settings to activate Windows.

Column Reading

KEYP:

	C3	C2	C1	C0
	1	1	1	1
0	3	2	1	0
0	7	6	5	4
0	B	A	9	8
0	F	E	D	C
	1	1	0	1

```
MOV AL, 0          ;AL = 0
OUT PA, AL         ;send zeros to all the rows
IN AL, PB          ;read in the columns
AND AL, 0FH        ;mask the upper nibble
CMP AL, 0FH        ;compare with 0FH
JZ KEYP            ;if equal, no key, keep checking
CALL DELAY         ;if key press, wait for debounce

IN AL, PB          ;read in the columns
AND AL, 0FH        ;mask the upper nibble
CMP AL, 0FH        ;compare with 0FH
JZ KEYP            ;if equal, no key, check again
                   ;Next, ground one row at a time
```

	CMP AL,OFH							
PORT B	1	1	1	1	1	1	0	1
0FH	0	0	0	0	1	1	1	1
CMP	Upper Nibble Masked				Not Equal			

Activate Windows
Go to Settings to activate Windows.

Row Reading - Row 0

```
ROW0:   MOV AL, 0FEH     ;send '0' to Row0 alone
        OUT PA, AL
        IN AL, PB        ;read in the columns
        AND AL, 0FH      ;mask the upper nibble
        CMP AL, 0FH      ;compare with 0FH
        JZ ROW1          ;no keypress in Row0, check Row1
        LEA SI, ROW_0    ;not equal, SI to point to Row0 data
        JMP COL_ID       ;go to finding the column
```

		C3	C2	C1	C0
		1	1	1	1
R0	0	3	2	1	0
R1	1	7	6	5	4
R2	1	B	A	9	8
R3	1	F	E	D	C
		1	1	1	1
		PB3	PB2	PB1	PB0
		C3	C2	C1	C0

AL	F				E			
	1	1	1	1	1	1	1	0
					R3	R2	R1	R0

	CMP AL,0FH							
PORT B	1	1	1	1	1	1	1	1
0FH	0	0	0	0	1	1	1	1
CMP	Low nibble Masked				Equ			

12:24 / 20:23 • Row Reading - Row 0
Activate Windows
Go to Settings to activate Windows.

Row Reading - Row 1

```
ROW1:   MOV AL, 0FDH    ;send '0' to Row1 alone
        OUT PA, AL      ;repeat steps
        IN AL, PB
        AND AL, 0FH
        CMP AL, 0FH
        JZ ROW2
        LEA SI, ROW_1
        JMP COL_ID
```

		C3	C2	C1	C0
		1	1	1	1
R0	1	3	2	1	0
R1	0	7	6	5	4
R2	1	B	A	9	8
R3	1	F	E	D	C
		1	1	1	1
		C3	C2	C1	C0
		PB3	PB2	PB1	PB0

AL	F				D			
	1	1	1	1	1	1	0	1
					R3	R2	R1	R0

	CMP AL,OFH							
PORT B	1	1	1	1	1	1	1	1
0FH	0	0	0	0	1	1	1	1
CMP	Upper Nibble Masked				Equal			

14:19 / 20:23 • Row Reading - Row 0

Row Reading - Row 2

```
ROW2:   MOV AL, 0FBH    ;send '0' to Row2 alone and
        OUT PA, AL      ;repeat steps
        IN AL, PB
        AND AL, 0FH
        CMP AL, 0FH
        JZ ROW3
        LEA SI, ROW_2
        JMP COL_ID
```

SI points to the ROW 2 Data Address :6009

Go to finding the column

		C3	C2	C1	C0
		1	1	1	1
R0	1	3	2	1	0
R1	1	7	6	5	4
R2	0	B	A	9	8
R3	1	F	E	D	C
		1	1	0	1
		C3	C2	C1	C0
		PB3	PB2	PB1	PB0

AL	F				B			
	1	1	1	1	1	0	1	1
					R3	R2	R1	R0

	CMP AL,OFH							
Port B	1	1	1	1	1	1	0	1
0F	0	0	0	0	1	1	1	1
CMP	Upper Nibble Masked				N Equ			

Activate Windows
Go to Settings to activate Windows.

15:49 / 20:23 • Row Reading - Row 0

Row Reading - Row 3

```
ROW3:   MOV AL, 0F7H    ;send '0' to Row3 alone and
        OUT PA, AL      ;repeat steps
        IN AL, PB
        AND AL, 0FH
        CMP AL, 0FH
        JZ KEYP
        LEA SI, ROW_3
```

Row Reading - Row 2

		C3	C2	C1	C0
		1	1	1	1
R0	1	3	2	1	0
R1	1	7	6	5	4
R2	0	B	A	9	8
R3	1	F	E	D	C
		1	1	0	1
		C3	C2	C1	C0
		PB3	PB2	PB1	PB0

```
ROW2:   MOV AL, 0FBH    ;send '0' to Row2 alone and
        OUT PA, AL      ;repeat steps
        IN AL, PB
        AND AL, 0FH
        CMP AL, 0FH
        JZ ROW3
        LEA SI, ROW_2
        JMP COL_ID
```

AL	F				B			
	1	1	1	1	1	0	1	1
					R3	R2	R1	R0

SI points to the ROW 2 Data Address :6009

Go to finding the column

	CMP AL,0FH							
Port B	1	1	1	1	1	1	0	1
0F	0	0	0	0	1	1	1	1
CMP	Masked				N	Equ		

17:09 / 20:23 • Row Reading - Row 0
Activate Windows
Go to Settings to activate Windows.

Key Press Identification

```
COL_ID: SHR AL, 1        ;to identify column, shift right
        JNC FOUND        ;if no carry, column is found
        INC SI           ;otherwise point SI to next row
        JMP COL_ID       ;go back to finding the column
FOUND:  MOV AL, [SI]     ;get data pointed by SI into AL
        MOV KEY, AL      ;this is the pressed key. Store in KEY
    DELAY PROC NEAR
        MOV CX, 6000     ;delay for key debounce
AGN:    LOOP AGN
        RET
  DELAY   ENDP
          END
```

	SI				
ROW 0	6000	0	1	2	3
ROW 1	6005	4	5	6	7
ROW 2	6009	8	9	A	B
ROW 3	600D	C	D	E	F

17:49 / 20:23 • Row Reading - Row 0

Row Reading - Row 2

```
ROW2:   MOV AL, 0FBH    ;send '0' to Row2 alone and
        OUT PA, AL      ;repeat steps
        IN AL, PB
        AND AL, 0FH
        CMP AL, 0FH
        JZ ROW3
        LEA SI, ROW_2
        JMP COL_ID
```

		C3	C2	C1	C0
		1	1	1	1
R0	1	3	2	1	0
R1	1	7	6	5	4
R2	0	B	A	9	8
R3	1	F	E	D	C
		1	1	0	1
		C3	C2	C1	C0
		PB3	PB2	PB1	PB0

AL	F				B			
	1	1	1	1	1	0	1	1
					R3	R2	R1	R0

SI points to the ROW 2 Data Address :6009

Go to finding the column

	CMP AL,OFH							
Port B	1	1	1	1	1	1	0	1
0F	0	0	0	0	1	1	1	1
Clr	Upper Nibble Masked				Not Equal			

18:04 / 20:23 • Row Reading - Row 2

Key Press Identification

```
COL_ID: SHR AL, 1        ;to identify column, shift right
        JNC FOUND        ;if no carry, column is found
        INC SI           ;otherwise point SI to next row
        JMP COL_ID       ;go back to finding the column
FOUND:  MOV AL, [SI]     ;get data pointed by SI into AL
        MOV KEY, AL      ;this is the pressed key. Store in KEY
    DELAY PROC NEAR
        MOV CX, 6000     ;delay for key debounce

AGN:    LOOP AGN
        RET
  DELAY   ENDP
          END
```

	SI				
ROW 0	6000	0	1	2	3
ROW 1	6005	4	5	6	7
ROW 2	6009	8	9	A	B
ROW 3	600D	C	D	E	F

1 1 0 1

18:14 / 20:23 • Row Reading - Row 2 >

Defining CWR , Port A and Port B of 8255

8255 PPI Programming							
Control Word Register (0C6H)							
8				2			
1	0	0	0	0	0	1	0
I/O	MODE - 0		PORT A		MODE - 0	PORT B	
	PORT A		O/P		PORT B	I/P	

```
ROW_0 DB 0, 1, 2, 3
ROW_1 DB 4, 5, 6, 7
ROW_2 DB 8, 9, 0AH, 0BH
ROW_3 DB 0CH, 0DH, 0EH, 0FH
KEY DB 0

   CR EQU 0C6H      ;address of the control register
   PA EQU 0C0H      ;address of Port A
   PB EQU 0C2H      ;address of Port B

   MOV AL, 82H      ;8255 control word
   OUT CR, AL       ;send to control register
```

Activate Windows
Go to Settings to activate Windows.

Key Press Identification

```
COL_ID: SHR AL, 1        ;to identify column, shift right
        JNC FOUND        ;if no carry, column is found
        INC SI           ;otherwise point SI to next row
        JMP COL_ID       ;go back to finding the column
FOUND:  MOV AL, [SI]     ;get data pointed by SI into AL
        MOV KEY, AL      ;this is the pressed key. Store in KEY
DELAY PROC NEAR
        MOV CX, 6000     ;delay for key debounce
AGN:    LOOP AGN
        RET
DELAY   ENDP
        END
```

	SI				
ROW 0	6000	0	1	2	3
ROW 1	6005	4	5	6	7
ROW 2	6009	8	9	A	B
ROW 3	600D	C	D	E	F

1 1 0 1

20:04 / 20:23 • Row Reading - Row 2 >

600... Key